beCook / Roadmap único de implementación

SEGURIDAD · DATOS · EXPERIENCIA · APERTURA A CLIENTES

Orden de ejecución: seguir los pasos numerados; cada uno debe implementarse y validarse antes de darlo por cerrado. Los cambios de interfaz se apoyan en contratos y operaciones de backend estabilizados en fases anteriores.

Prioridades: BLOQUEANTE = necesario antes de abrir el registro al público; ALTA = imprescindible para completar el recorrido de producto propuesto; MEDIA = mejora de claridad o mantenibilidad que puede cerrarse después de los bloqueantes.

Base: "Auditoría integral y roadmap" (backend; estado del código de 10/09/2026) y "Auditoría UI/UX beCook" (septiembre de 2026). Antes de cada paso, comprobar que el defecto siga presente en la rama actual. Las tareas expresamente marcadas "Extensión de lanzamiento" amplían los informes para preparar la apertura a clientes.

FASE 0 Contener riesgos y preparar un entorno seguro

01 BLOQUEANTE · Retirar credenciales de recuperación expuestas

Problema. La auditoría de backend detectó códigos de recuperación de Vercel en archivos seguidos por Git; eliminar el archivo no invalida las copias del historial.

Corrección. Verificar que existe otro método de acceso a la cuenta, revocar o regenerar los códigos desde Vercel, guardar los nuevos fuera del repositorio, retirar el archivo del seguimiento y añadir exclusiones y escaneo de secretos. Revisar si hay que sanear el historial y tratar cualquier otro secreto detectado sin publicarlo en logs ni prompts.

Referencia: Backend H1.

02 BLOQUEANTE · Preparar base de datos de pruebas y cambios reproducibles

Problema. El esquema real de Supabase no se reconstruye por completo con las migraciones versionadas y las pruebas de escritura pueden afectar a datos reales.

Corrección. Crear un entorno Supabase aislado con datos sintéticos; documentar y versionar una línea base del esquema, RLS y migraciones incrementales, sin ejecutar un reset sobre producción. Configurar una comprobación automática inicial de instalación, lint, tipos, tests y build y una revisión de migraciones antes de aplicarlas.

Referencia: Backend §12, §14.

03 BLOQUEANTE · Parchar dependencias vulnerables sin cambiar de arquitectura

Problema. La auditoría encontró avisos de seguridad en el lockfile, incluido Next, y el estado de los parches puede haber cambiado desde septiembre de 2026.

Corrección. Reauditar las versiones instaladas, actualizar selectivamente Next y sus dependencias relacionadas, Supabase Auth y transitivas afectadas según avisos vigentes; conservar el lockfile y comprobar compatibilidad, login, rutas, tests y build. Evitar actualizaciones indiscriminadas o una migración de stack.

Referencia: Backend H4.

04 BLOQUEANTE · Proteger las tres rutas de IA de pago

Problema. Generar menú, importar y generar compra podían invocarse sin verificación de sesión en el servidor; las políticas RLS no protegen esas llamadas y un invitado podía consumir la API.

Corrección. Definir explícitamente la política de invitados: lectura y edición local, pero IA solo con sesión verificada, salvo presupuesto invitado diseñado y aprobado. Verificar la identidad y propiedad en el servidor para los tres endpoints, enviar credenciales de sesión de forma segura, validar tamaños y tipos de entrada y rechazar solicitudes no autorizadas antes de llamar a OpenAI.

Referencia: Backend H2.

05 BLOQUEANTE · Establecer límites de gasto y trazabilidad fiable de IA

Problema. La telemetría se escribe desde el navegador, no sirve como contabilidad autorizada; faltan cuotas duraderas y límites de concurrencia para los tres flujos.

Corrección. Aplicar cuotas persistentes por usuario, límite de solicitudes simultáneas y protección contra abuso por IP, con respuestas claras ante límite o error. Registrar en servidor intento, flujo, modelo, resultado, coste estimado, latencia y request ID; evitar registrar perfiles, platos o textos libres sensibles. Extender timeouts y reintentos acotados a compra e importación y comprobar que cancelar la interfaz no implica prometer que se cancela un cobro ya iniciado.

Referencia: Backend H2, §9-10.

FASE 1 Garantizar identidad, guardado y coherencia entre dispositivos

06 BLOQUEANTE · Aislar datos locales por identidad y sesión

Problema. Las claves locales de perfil, despensa, semana y compra no distinguen usuarios; la caché de una cuenta puede mostrarse a otra o bloquear la lectura de datos remotos más recientes.

Corrección. Resolver identidad antes de cargar información, separar invitado y cada userId, invalidar vistas y cachés al cambiar de sesión y revalidar el perfil remoto. Ofrecer importación explícita de datos locales antiguos sin adjudicárselos automáticamente a ninguna cuenta. Probar A → cerrar sesión → B y uso en varios dispositivos.

Referencia: Backend H3.

07 BLOQUEANTE · Establecer una fuente de verdad y una política de sincronización

Problema. Perfil, semana, compra y despensa priorizan con frecuencia información local; lo guardado en Supabase no siempre se vuelve a leer y los cambios de otro dispositivo pueden quedar invisibles.

Corrección. Para usuarios autenticados, usar datos remotos como referencia canónica y cachés locales identificadas por usuario, entidad y versión. Definir resolución explícita de conflictos y tratamiento de datos pendientes; recuperar perfil, semana, compra y despensa remotos antes de declarar un estado vacío. Mantener modo local de invitado sin insinuar sincronización.

Referencia: Backend H3, H7, §3.

08 BLOQUEANTE · Reparar el contrato entre perfil y base de datos

Problema. El formulario admite 6–7 días de entrenamiento que el CHECK de la tabla rechaza, «Otro» no cabe en el campo legacy y los alimentos rechazados pierden información al releerse.

Corrección. Crear adaptadores y tipos alineados con el esquema real, corregir lectura/escritura de listas y objetivo legacy, ampliar de forma no destructiva el CHECK cuando proceda y probar guardar → recargar → leer desde otro dispositivo. No eliminar columnas ni cambiar requisitos de formulario antes de resolver este contrato.

Referencia: Backend H5; UX 02-04.

09 ALTA · Corregir calendario ISO y selección de semana

Problema. Fechas de vista previa fijas, importaciones parciales fechadas por posición, semanas 53 inválidas y flechas que no actualizan la URL pueden llevar a mostrar o guardar el menú en un periodo equivocado.

Corrección. Calcular todas las fechas desde año/semana ISO y día real, validar semanas 53 según el año y sincronizar semana seleccionada, anterior/siguiente/hoy y URL. Cubrir importaciones parciales, fin de año y recargas antes de volver a utilizar fechas en identificadores o firmas de caché.

Referencia: Backend §5, §9; UX 12-15.

10 BLOQUEANTE · Hacer seguras las escrituras de semana y menú

Problema. La aplicación reemplaza días con operaciones delete → insert separadas, puede intercalar ediciones concurrentes y dejar un menú parcialmente guardado.

Corrección. Actualizar individualmente los platos editados y aplicar/reemplazar semanas completas mediante una operación transaccional con control de propietario, versión esperada e idempotencia. Proteger doble clic y concurrencia, y conservar intacta la versión anterior cuando falle cualquier paso; permitir repetir una semana anterior sin saltarse estas garantías.

Referencia: Backend H8; UX 08, 15.

11 BLOQUEANTE · Separar aplicar un menú de generar su compra y mostrar guardado veraz

Problema. «Usar este menú» puede mostrar éxito aunque falle Supabase y espera una generación de compra que puede fallar o retrasarse; el usuario no sabe qué parte terminó.

Corrección. Devolver estados explícitos y separados de menú local, menú remoto y compra; abrir la semana tras confirmar el guardado del menú sin esperar la compra. Mantener propuesta recuperable y permitir reintentar únicamente la escritura fallida sin repetir llamadas de IA. No informar «Guardado en tu cuenta» hasta confirmación remota.

Referencia: Backend H9; UX 25.

12 BLOQUEANTE · Vincular cada lista de la compra a la semana correcta

Problema. La compra puede guardarse contra la última semana creada, no contra la elegida; al marcar productos se crean listas completas nuevas y un segundo dispositivo no recupera la lista existente.

Corrección. Utilizar weekId, listId e itemId estables, recuperar primero la lista remota de esa semana y actualizar solo el artículo modificado. Proteger escrituras simultáneas y conservar registros antiguos hasta definir una limpieza segura. Comprobar que marcar en una semana pasada no altera la actual ni crea otra lista.

Referencia: Backend H7; UX 14, 16.

13 BLOQUEANTE · Dar persistencia segura y recuperación remota a la despensa

Problema. La despensa se carga localmente y cada modificación remota reemplaza el conjunto mediante borrado e inserción; un fallo puede eliminar o desordenar artículos.

Corrección. Leer la despensa del usuario autenticado desde Supabase; crear, actualizar y borrar por artículo con identidad estable, comprobación de propietario, manejo de versiones y errores. Evitar reemplazos completos y duplicados; probar dos dispositivos y fallos de red antes de rediseñar los controles visuales.

Referencia: Backend H8, §8; UX 18-20.

14 ALTA · Evitar compras regeneradas por cambios irrelevantes

Problema. La firma actual incorpora fechas mostradas y una caché única; navegar o aplicar un menú puede provocar otra llamada de IA y perder estados de compra aunque los platos no cambien.

Corrección. Construir la firma a partir de identidad y versión de la semana, platos, raciones, restricciones y versión del origen de ingredientes, nunca de texto de presentación. Reutilizar la compra guardada al recargar y separar regeneración de ingredientes de un simple marcado comprado.

Referencia: Backend §5, §9, §13; UX 14.

15 BLOQUEANTE · Recuperar escrituras fallidas y hacer visible la sincronización

Problema. Los fallos remotos se ocultan en warnings y los datos locales pueden aparentar estar a salvo aun cuando no se han sincronizado.

Corrección. Exponer estados «Guardado en tu cuenta», «Solo en este dispositivo», «Pendiente de sincronizar» y «Error: reintentar» según el resultado real. Recuperar cambios pendientes por entidad, evitar sobrescrituras y no disparar IA para reintentar una escritura. Mostrar el estado coherentemente en semana, compra, despensa y perfil.

Referencia: Backend H9; UX 25.

FASE 2 Crear un contrato culinario único antes de rediseñar los flujos

16 BLOQUEANTE · Congelar el contexto alimentario de cada semana

Problema. El generador usa información de perfil desde el navegador, mientras que la compra recibe solo nombres de platos: presupone dos personas y pierde alergias, restricciones y preferencias.

Corrección. Cargar y validar en servidor el perfil de la identidad autenticada y registrar por semana raciones y una versión explícita de restricciones/preferencias utilizadas. Transmitir ese contexto a generación, vista previa, recetas y compra; mostrar cuándo un cambio posterior del perfil requiere revisar la semana existente. No afirmar compatibilidad alimentaria a partir de un título.

Referencia: Backend H6; UX 10, 11.

17 BLOQUEANTE · Crear fichas de plato versionadas que alimenten menú y compra

Problema. Los platos solo persisten como nombres; no hay ingredientes, cantidades, tiempo ni preparación. La compra intenta adivinar ingredientes en una llamada independiente.

Corrección. Definir identidad y versión de receta/plato con ingredientes, unidades, cantidad base, raciones, tiempo, pasos y posibles alérgenos identificados. Hacer que cada slot de semana apunte a la versión concreta; ofrecer adaptación explícita para menús históricos con solo título y no presentar como verificados los ingredientes inferidos. Reutilizar estas fichas en la futura pantalla de cocina y en la compra.

Referencia: Backend H6, fase 3; UX 11.

18 BLOQUEANTE · Validar ingredientes y restricciones antes de usar las recetas

Problema. Un nombre de plato no revela ingredientes ocultos; el filtro por palabras del título no detecta todas las alergias, incluidos casos introducidos como texto libre.

Corrección. Comprobar restricciones contra ingredientes estructurados y advertir o bloquear propuestas incompatibles detectadas, incluidas variantes de nombres y restricciones libres que el sistema no pueda interpretar con seguridad. Mantener una distinción visible entre ingredientes inferidos y revisados; no prometer ausencia certificada de alérgenos ni dietas terapéuticas.

Referencia: Backend H6, §10.

19 BLOQUEANTE · Construir la compra desde las mismas recetas, raciones y despensa

Problema. Menú y compra pueden producir cantidades o ingredientes distintos; «En despensa» no distingue lo que falta, y no se conoce cuánto stock existe en casa.

Corrección. Agregar ingredientes de fichas versionadas para las raciones de la semana, normalizar unidades compatibles y asociar cada artículo a su origen. Separar «Te falta comprar», «Ya lo tienes habitualmente» y «Comprado» sin descontar cantidades de despensa que no se han medido. Permitir revisar manualmente cantidades y artículos no reconocidos; no anunciar ahorros exactos sin una base comprobable.

Referencia: Backend H6; UX 17, 19-20.

20 BLOQUEANTE · Reconciliar cambios de plato sin borrar lo ya comprado

Problema. Editar un plato invalida la lista completa: puede reiniciarse la compra y perderse artículos marcados; conservar checks solo por nombre también puede ocultar cambios de cantidad.

Corrección. Tras editar o sustituir un slot, calcular únicamente el delta de ingredientes y actualizar la lista existente por identidad estable. Conservar el historial de artículos comprados, distinguir adiciones, eliminaciones y cantidades modificadas y pedir revisión cuando un check anterior deje de equivaler a la nueva necesidad. Mostrar un resumen claro del cambio sin recrear listas ni facturar otra semana entera.

Referencia: Backend H7-H8; UX 06, 14.

21 ALTA · Aplicar reglas culinarias y sustitución selectiva

Problema. El validador admite salsas ambiguas como «salsa especial» y el usuario debe regenerar toda la semana para descartar un solo plato.

Corrección. Rechazar títulos culinariamente indeterminados con reglas y pruebas que admitan salsas concretas válidas. Añadir operación de «otra idea» limitada a un slot: seleccionar alternativa compatible revisada o hacer un único intento acotado, revalidar el menú completo y conservar los otros trece platos. Registrar coste y errores; no arreglar un título inventando ingredientes.

Referencia: Backend H10, §10; UX 06.

22 ALTA · Endurecer importación y recuperación de propuestas

Problema. La importación puede enviar tablas claras a la IA innecesariamente, admite entradas poco acotadas y muestra errores genéricos; la propuesta de menú vive solo en memoria y puede aplicarse desactualizada.

Corrección. Parsear formatos inequívocos localmente y recurrir a IA para texto ambiguo, con límites de entrada, salida y respuestas 400/422/502 diferenciadas. Guardar borradores por usuario y semana, vincular cada propuesta a su contexto y bloquear la aplicación de una propuesta antigua mientras se genera otra. Para menús importados, advertir incompatibilidades sin modificar silenciosamente la elección del usuario.

Referencia: Backend §9; UX 07-09.

FASE 3 Diseñar una experiencia continua sobre las capacidades ya estables

23 ALTA · Completar alta, login y recuperación de cuenta

Problema. El login dirige siempre a Perfil, la propuesta de valor es poco clara y no existe un recorrido de recuperación de contraseña/callback verificado con una cuenta real.

Corrección. Implementar y probar registro, confirmación cuando corresponda, recuperación de contraseña y retorno al destino solicitado; explicar guardado entre dispositivos sin lenguaje técnico de Supabase. Resolver una sesión caducada sin perder borradores locales ni enviar al usuario a un bucle de redirección.

Referencia: Backend §8; UX 01.

24 ALTA · Simplificar el perfil y eliminar datos personales inventados

Problema. El perfil preselecciona sexo, altura, peso y dos comensales sin confirmación; mezcla ajustes, preferencias y restricciones en un formulario largo, y algunos datos corporales no llegan al generador.

Corrección. Mostrar primero comensales, objetivos, alergias y restricciones con confirmación explícita de «ninguna»; guardar por bloques y señalar información pendiente o asumida. Posponer campos corporales no usados, o mantenerlos opcionales tras adaptar el esquema; si se mantienen altura y peso, usar entrada numérica móvil con unidades. Enseñar qué preferencias afectan realmente al menú.

Referencia: UX 02-04; Backend H5, fase 3.

25 ALTA · Implementar onboarding de tres pasos y primer menú

Problema. Una cuenta nueva aterriza en una semana vacía, sin descubrir perfil ni despensa; crear el primer menú exige que el usuario conozca previamente la navegación de la aplicación.

Corrección. Crear el recorrido «quién eres y qué ofrece beCook → comensales, objetivo y restricciones → despensa exprés opcional», con básicos sin seleccionar por defecto, guardado por paso y reanudación entre dispositivos. Distinguir cuentas previas de altas nuevas; al terminar, generar automáticamente una propuesta con el contexto ya persistido y pedir confirmación antes de aplicarla. Si falla la IA, reintentar sin repetir el alta.

Referencia: UX 01, 02, 04, 19; Backend H2-H3, H5-H6.

26 ALTA · Hacer segura y visible la aplicación de una propuesta

Problema. «Usar este menú» queda oculto después de siete tarjetas y el reemplazo utiliza un diálogo del navegador que no explica el alcance de la operación.

Corrección. Mostrar CTA primario fijo con resumen de la propuesta y confirmación propia que identifique semana, menú afectado y consecuencias para la compra. Ofrecer deshacer breve solo si se puede restaurar de forma consistente el estado anterior, con versiones y control de cambios concurrentes; no prometer recuperar checks si ya se han producido acciones irreversibles.

Referencia: UX 05, 08; Backend H8-H9.

27 ALTA · Representar con honestidad la generación y sus reintentos

Problema. La barra de progreso actual aparenta avanzar sin medir progreso real, no explica la espera y perder la pantalla puede hacer desaparecer la propuesta.

Corrección. Mostrar etapas solo cuando el servidor pueda confirmarlas; en otro caso usar un indicador indeterminado y rango orientativo, no porcentajes inventados. Añadir cancelación con efecto real y explicación de sus límites, borrador recuperable, reintento acotado y resumen visible «generando para…» con comensales y restricciones verificadas.

Referencia: UX 07, 10; Backend §9-10.

28 ALTA · Permitir sustituir un plato desde la vista previa

Problema. La propuesta obliga a aceptar los catorce platos o regenerar toda la semana, incluso cuando solo uno no encaja.

Corrección. Incorporar «Otra idea» en cada plato, apoyándose en el servicio de sustitución selectiva, recetas versionadas y conciliación de compra ya implementados. Conservar los demás platos, impedir resultados incompatibles y ofrecer un estado claro de generación/error; posponer bloqueo de días hasta tener evidencia de su necesidad.

Referencia: UX 06; Backend H10.

29 ALTA · Abrir Semana con un bloque destacado de Hoy

Problema. Siete tarjetas equivalentes dificultan encontrar la comida y cena del día y los colores actuales compiten entre «hoy» y «día seleccionado».

Corrección. Destacar comida y cena de hoy y mostrar los restantes días compactos, con una distinción visual consistente entre día actual y selección. Mantener el calendario ISO y la semana elegida como fuente de verdad, resolver títulos estrechos y dar acceso a la ficha de cada plato, edición y guardado por slot.

Referencia: UX 11-13; Backend §8.

30 ALTA · Mostrar la compra completa y el valor real de la despensa

Problema. Solo se ve una categoría por vez y los artículos disponibles en casa siguen mezclados con lo pendiente; la cabecera ocupa demasiado espacio en móvil.

Corrección. Mostrar una lista continua con secciones y accesos rápidos, progreso global y distinción entre pendiente, comprado y «Ya lo tienes habitualmente», con este último grupo plegado. Mostrar semana y raciones, permitir revisar la omisión de un artículo y compactar la cabecera al desplazarse sin ocultar controles de compra.

Referencia: UX 16-17, 24; Backend H6-H7.

31 ALTA · Dar a Despensa acciones claras y edición segura de la compra

Problema. La casilla de Despensa significa algo distinto de la de Compra, rellenar básicos requiere altas repetidas y no se pueden añadir artículos ajenos al menú o ajustar cantidades.

Corrección. Usar un control explícito «Tengo / Se acabó» en despensa, selección múltiple de básicos y alta rápida desde un artículo comprado; permitir añadir artículos manuales y ajustar cantidades en Compra. Confirmar o permitir deshacer borrados de despensa con la operación remota ya protegida; no simular existencias exactas.

Referencia: UX 18-20; Backend §8.

32 MEDIA · Simplificar navegación, importación y estados vacíos

Problema. «Crear menú» ocupa navegación permanente aunque sea una acción esporádica; importar pesa visualmente lo mismo que generar y una semana vacía solo muestra un botón.

Corrección. Concentrar la navegación en Semana, Compra y Perfil; ofrecer Crear menú contextual en Semana y acceso directo a Despensa desde Compra. Presentar importar como alternativa secundaria, y en semana vacía ofrecer crear o repetir la anterior con información clara sobre lo que ocurrirá. Conservar redirecciones antiguas durante una transición y eliminarlas solo tras comprobar que no tienen consumidores.

Referencia: UX 09, 15, 21, 26.

33 ALTA · Corregir legibilidad, interacción táctil y accesibilidad

Problema. Hay textos y chips pequeños, contraste insuficiente en algunos CTAs, foco invisible, controles sin etiqueta contextual y roles de pestaña incompletos; faltan comprobaciones con teclado móvil y lector de pantalla.

Corrección. Subir los textos de lectura habitual a un tamaño cómodo y diseñar objetivos táctiles cercanos a 44 px donde corresponda; corregir contraste y foco, labels, avisos accesibles y semántica real de grupos/tabs. Probar a 320/390 px, Safari/iPhone físico, teclado, safe-area y VoiceOver; conservar filas de compra amplias y la paleta visual existente.

Referencia: UX 23, 26; Backend §8.

34 MEDIA · Consolidar componentes y mejorar cabeceras sin reescribir pantallas

Problema. Conviven componentes propios, wrappers shadcn y clases/escala tipográfica duplicadas, lo que provoca deriva visual; cabeceras pegajosas reducen el espacio útil.

Corrección. Auditar consumidores reales y consolidar botones y tipografía por migraciones pequeñas y verificadas, sin borrar wrappers todavía usados. Uniformar anchos de formularios, mensajes de estado y cabeceras compactables; eliminar rutas y código heredados únicamente con pruebas de navegación y build satisfactorias.

Referencia: UX 22, 24, 26; Backend §14.

FASE 4 Validar el producto y abrirlo de forma controlada

35 BLOQUEANTE · Cubrir los recorridos críticos con pruebas automatizadas

Problema. Los siete tests observados se concentran en IA; no protegen login real, sincronización, RLS de escritura, recuperación ni el flujo completo del primer menú.

Corrección. Añadir pruebas de contrato de base de datos y RLS con usuarios A/B, integración de los tres endpoints y pruebas E2E de registro → onboarding → propuesta → semana → receta → compra. Cubrir duplicados, fallo de red, dos dispositivos, edición, reintento y migración. Ejecutarlas en CI sobre entorno aislado con lint, tipos y build; no dirigir escrituras de test a producción.

Referencia: Backend §12; UX 01-26.

36 BLOQUEANTE · Verificar usabilidad real, rendimiento y errores en dispositivos

Problema. Las auditorías no probaron una sesión completa en iPhone físico ni aportan datos de uso o Core Web Vitals de producción; un build correcto no demuestra que la experiencia funcione en campo.

Corrección. Completar pruebas manuales de tareas reales en Safari/iPhone y Android, pantallas estrechas, redes lentas, teclado y accesibilidad. Medir LCP/CLS/INP y tiempo hasta menú útil en rutas clave; corregir problemas reproducibles de navegación, disponibilidad, bloqueo y tiempos antes de abrir al público.

Referencia: Backend §8, §13; UX 23-24.

37 BLOQUEANTE · Preparar operación, privacidad y recuperación ante fallos

Problema. La auditoría no valida backups, restauración, alertas de costes, gestión de datos de usuarios, documentación de despliegue ni comportamiento frente a incidencias de producción.

Corrección. Configurar alertas de errores y presupuesto OpenAI, revisar logs sin datos alimentarios sensibles, probar copia/restauración y rollback de aplicación y migraciones, documentar variables/entornos y respuesta a incidentes. Preparar y revisar textos públicos de privacidad y condiciones y procesos de acceso, exportación y eliminación de cuenta/datos antes del registro abierto. Verificar con responsables competentes las obligaciones aplicables; estas tareas son una ampliación de preparación para lanzamiento.

Referencia: Extensión de lanzamiento; Backend §11-14.

38 BLOQUEANTE · Ejecutar beta cerrada y abrir por etapas con criterios de salida

Problema. Las dos auditorías son revisiones de código y sesiones limitadas; no demuestran que usuarios reales completen los flujos ni que el sistema resista nuevas altas sin fallos.

Corrección. Probar con un grupo pequeño de cuentas nuevas y existentes y registrar abandonos, errores, sincronización, coste de IA y fallos culinarios. Corregir bloqueantes, repetir regresión y abrir gradualmente solo tras verificar credenciales y endpoints protegidos, aislamiento A/B, datos consistentes en varios dispositivos, recetas y compra coherentes, flujos E2E y procedimientos de rollback operativos. No interpretar «sin errores» como garantía absoluta: definir detección y recuperación.

Referencia: Extensión de lanzamiento; Backend §12-13; UX 01-26.

Regla de cierre: un paso no se considera completado por el mero hecho de que el código compile: requiere comprobar su comportamiento y que no rompe los recorridos ya existentes. Ninguna migración destructiva ni prueba de escritura debe ejecutarse directamente sobre datos de producción.